I have a web server running using the Python framework FastAPI. But I have a problem. Sometimes, certain requests require a lot of computation and end up running a long time. I don't want users to have to wait such a long time to get a response back from the server. I want the server to cancel requests that take longer than 5 seconds. Do you know of a way to do that?

Yes, you can implement a timeout mechanism for your FastAPI application to cancel requests that take longer than a specified duration. FastAPI itself doesn't provide a built-in timeout feature, but you can use Python's `asyncio` to achieve this.

Here's an example of how you can implement a timeout for requests in FastAPI:

python

```python
from fastapi import FastAPI, HTTPException
import asyncio

app = FastAPI()

async def perform_computation():
    # Simulate a long-running computation
    await asyncio.sleep(10)
    return "Computation complete"

@app.get("/compute")
async def compute():
    try:
        result = await asyncio.wait_for(perform_computation(), timeout=5)
        return {"result": result}
```

Copy code

```
    except asyncio.TimeoutError:
        raise HTTPException(status_code=500, detail="Request timed out")
```

In this example:

1. The `perform_computation` function simulates a long-running computation using `asyncio.sleep`.
2. The `/compute` route uses `asyncio.wait_for` to set a timeout of 5 seconds for the `perform_computation` function.
3. If the computation takes longer than 5 seconds, a `asyncio.TimeoutError` will be caught, and a `HTTPException` with a 500 status code and a "Request timed out" message will be raised.

You can customize the timeout duration and the error message based on your requirements.

Note: Ensure that your server is configured to handle asynchronous requests by running it with an ASGI server like uvicorn:

bash

```
uvicorn your_app:app --host 0.0.0.0 --port 8000 --reload
```

Copy code

Replace `your_app` with the filename of your FastAPI application.

How would you make it such that this timeout code ran on every single one of my routes?

ChatGPT 5.2

middleware. FastAPI allows you to define custom middleware functions that run for every request before it reaches the route handler.

Here's an example of how you can create a middleware to handle timeouts for all routes:

python

Copy code

```python
from fastapi import FastAPI, HTTPException
from fastapi.middleware.trustedhost import TrustedHostMiddleware
import asyncio

app = FastAPI()

# Middleware to handle timeouts
async def timeout_middleware(request, call_next):
    try:
        # Set a timeout for the entire request handling process (including
        response = await asyncio.wait_for(call_next(request), timeout=5)
        return response
    except asyncio.TimeoutError:
        raise HTTPException(status_code=500, detail="Request timed out")

# Add the middleware to the FastAPI app
app.add_middleware(timeout_middleware)

# Your routes and handlers go here
@app.get("/compute")
async def compute():
    # Simulate a long-running computation
    await asyncio.sleep(10)
    return {"result": "Computation complete"}
```

In this example:

1. The `timeout_middleware` function is a middleware that sets a timeout for the entire request handling process, including the execution of route handlers.
2. The `call_next` function is used to proceed with the normal request handling. It represents the next middleware or the route handler.
3. If the entire request handling process takes longer than 5 seconds, an `asyncio.TimeoutError` is caught, and a `HTTPException` with a 500 status code and a "Request timed out" message is raised.
4. The middleware is added to the FastAPI app using `app.add_middleware(timeout_middleware)`.

Now, this middleware will be executed for every request, and it will enforce the specified timeout duration. Adjust the timeout duration as needed for your application.